

REPORT

01

NajdaAI

مساعد الطوارئ الطبي الذكي



تقرير الفريق — NajdaAI

تجربة المستخدم وقاعدة البيانات

UI/UX & Database

سري — للفريق فقط

٢٠ أغسطس ٢٠٢٦

2026-08-20

2026-08-20

NajdaAI



تقرير الفريق — NajdaAI

تجربة المستخدم وقاعدة البيانات

UI/UX & Database

المحتويات

نظرة عامة على تجربة المستخدم — الرحلة كاملة بالشاشات

1

نظام التصميم

2

مكونات الـ UX الحرجة

3

قاعدة البيانات

4

قصص حرب حقيقية (تدريبات مصداقية قدام اللجنة)

5

أسئلة متوقعة من الحكام + إجابات جاهزة

6

ده مرجع تحضير لتقديم المسابقة بكرة. المطلوب منك تكون قادر ترد على أي سؤال تفصيلي من اللجنة عن تجربة المستخدم وقاعدة البيانات، من غير ما ترجع للكود قدامهم. كل حاجة هنا اتقرأت من الكود الفعلي (مش افتراضات)، ومعها أرقام السطور والملفات علشان لو حد سألك "فين ده بالضبط" تقدر تشاور بثقة.

1 نظرة عامة على تجربة المستخدم — الرحلة كاملة بالشاشات

النظام كله React SPA واحد (`frontend/src/App.jsx`) بيستخدم `react-router-dom` (v7) بنظام `BrowserRouter` . الراوتس الأساسية:

المسار	الصفحة	ملاحظة
<code>/</code>	<code>Home.jsx</code>	هوم بيدج تسويقية (Hero + Boxes) — تحت <code>MainLayout</code> (فيه <code>Header</code>)
<code>/signup</code>	<code>SignUp.jsx</code>	تسجيل حساب جديد (فورم كامل)
<code>/login</code>	<code>Login.jsx</code>	تسجيل دخول (إيميل/باسورد + Google)
<code>/complete-profile</code>	<code>CompleteProfile.jsx</code>	البوابة — جهة اتصال الطوارئ إجبارية
<code>/profile</code>	<code>Profile.jsx</code>	عرض البروفايل الكامل
<code>/edit-profile</code>	<code>EditProfile.jsx</code>	تعديل البروفايل
<code>/chat</code>	<code>Chat.jsx</code>	الشات العادي، وبرضه وضع الطوارئ لو <code>?mode=emergency</code>
<code>/emergency</code>	<code>EmergencyMode.jsx</code>	صفحة تمهيدية لدخول الطوارئ (بلا تسجيل دخول)
<code>/emergency-auth</code>	<code>EmergencyAuth.jsx</code>	تسجيل دخول سريع بـ Google مخصص لمسار الطوارئ
<code>/emergency-history</code>	<code>EmergencyHistory.jsx</code>	سجل كل أحداث الطوارئ السابقة
<code>*</code>	—	أي مسار غير معروف يرجع لـ <code>/</code> (<code>Navigate replace</code>)

■ الرحلة الكاملة (Happy Path)

• 1. Signup / Google → الحساب

- التسجيل العادي (`SignUpFormFields.jsx`) بيعت لـ `POST /auth/register` مع كل بيانات الملف الطبي مرة واحدة (اختيارية إلا الإيميل/الباسورد/الاسم).
- تسجيل الدخول بجوجل (GSI — Google Identity Services، مش Firebase) بيعت لـ `POST /auth/google` ، والباك إند بيتحقق منه مع Google نفسها (`google.oauth2.id_token`) وبعدين بيطلع JWT بتاعنا. لو الإيميل مسجل local (باسورد) قبل كده بترجع 409 — مفيش دمج صامت بين طريقتين تسجيل لنفس الإيميل، ده قرار أمان مقصود.
- بعد أي طريقة تسجيل، الفرونت إند بيخزن الـ access token في `localStorage` (`setAccessToken`) في `frontend/src/lib/api.js` ويودي لـ `/profile` .

• 2. بوابة إكمال البيانات (Profile-Completion Gate)

- أول ما تدخل `/profile` أو أي صفحة محمية، `useEmergencyContactGate()`
- `frontend/src/hooks/useEmergencyContactGate.js` يعمل `GET /profile/me` مرة واحدة، ولو مفيش `emergency_contact.name` و `emergency_contact.phone` — سوا — يعمل `navigate("/complete-profile?next=" + <المسار الحالي>, { replace: true })`
- `/complete-profile` (`CompleteProfile.jsx`) فيها قسمين واضحين بصريًا:
 - **قسم 1 — إجباري:** اسم + رقم جهة اتصال الطوارئ (الإيميل اختياري). زرار "Save & Continue" مقفول (`disabled`) لحد ما الاسم والرقم يتملوا.
 - **قسم 2 — اختياري:** patient ID، تاريخ ميلاد، جنس، فصيلة دم، أمراض مزمنة. عليه زرار "Skip the rest" منفصل — بيحفظ جهة الاتصال بس ويكمل، من غير ما يجبرك تملى بيانات طبية مش لازم تديها دلوقتي.
 - الفشل في حفظ القسم الاختياري **مايقفش** الرحلة (`try/catch` منفصل في `handleSaveAndContinue`، سطر 271-280) — القسم الوحيد اللي لازم ينجح هو جهة الاتصال.
- **مهم جدًا:** البوابة دي بتتخطى بالكامل لو انت داخل مسار الطوارئ (`useEmergencyContactGate({ skip: isEmergencyMode })` في `Chat.jsx` سطر 77). التفصيلة دي أساسية لفهم فلسفة التصميم — شوف قسم 3.

• 3. الشات (العادي)

- `Chat.jsx` بيعمل session جديدة (`POST /chat/sessions`) أول ما تبعت أول رسالة، أو بيحمل session موجودة من ال query param `?session=<id>` (السايدبار `SideBar.jsx`) بيعرض هيستوري كل المحادثات (`GET /chat/sessions`)، والعنوان بيتحدد تلقائيًا من أول 40 حرف في أول رسالة (`chat.py`، `auto-title`، سطر 294-295).
- كل رسالة بتتبع ل `POST /chat/sessions/{id}/messages`، والرد ببيجي مع `sources[]` (روابط مصادر طبية) و `risk_level` (`low/moderate/high/emergency`) — الاتنين معروضين في الواجهة (`chips` للمصادر، `badge` ملون للخطورة).
- فيه إدخال صوتي (`Web Speech API`، `ar-EG`) ومكالمة حية `full-duplex` (`LiveCall.jsx` عبر `WebSocket` `/chat/live`) — دول مش من نطاق البريفنج ده لكن اعرف إنهم موجودين لو سألوا.

• 4. الطوارئ

- لو رسالة عادية في شات عادي طلعت `risk_level: "emergency"`، السيرفر نفسه بيفتح `EmergencyEvent` جديد بدون ما المستخدم يطلب — الشخص اللي مكانش حاسس إنه في خطر برضه بياخد نفس مسار الأمان (`chat.py` سطر 267-291، تعليق صريح في الكود بيشرح السبب).
- لو دخلت من `/emergency` (بدون حساب) → لازم تسجل دخول بجوجل بس (`/emergency-auth`) — مفيش فورم تسجيل طويل وقت أزمة.
- جوه وضع الطوارئ في `Chat.jsx`: بعد أول رد بمخاطرة `high/emergency`، بيظهر **كارت العداد التنازلي** (60 ثانية) + **سرينة صوت + اهتزاز** لحد ما تدوس "أنا بخير" أو الوقت يخلص ويتصعد تلقائيًا لجهة الاتصال. تفاصيل كاملة في قسم 3.

• 5. السجل (History)

- `/emergency-history` (`EmergencyHistory.jsx`) بيعرض كل الأحداث اللي حصلت للمستخدم: الحالة (`status` (`monitoring/alert_pending/resolved/escalated`), `stroke/chest_heart/breathing/unknown`), مستوى الخطورة،

ولينك يرجعك لنفس المحادثة اللي حصل فيها الحدث.

■ له فيه صفحتين تسجيل دخول (/login و /emergency-auth) ؟

عمدًا. /login هو المسار العادي (إيميل/باسورد + جوجل) اللي بيوديكي /profile . /emergency-auth مسار مختصر Google-only بيوديكي على طول /emergency — مفيش وقت وقت أزمة تدخل باسورد أو تختار. الاثنين بيستخدموا نفس ال GOOGLE_CLIENT_ID ونفس ال backend endpoint (POST /auth/google) — فرق الواجهة بس، مش فرق أمان.

2 نظام التصميم

■ 2.1 الألوان

اللون الأساسي هو الأخضر الزمردى #19A878 (وتنوعاته حسب السياق):

- #19A878 / #1AA681 — اللون الأساسي (أزرار، حدود focus، أيقونات نشطة)
 - #15966B / #168F68 / #148E70 — hover states أعمق شوية
 - #27B58A / #35C19B — لمسات في اللوجو والسايديبار
 - #E5F6F0 / #EAF8F4 / #F0FAF7 — خلفيات فاتحة جدًا (soft accent) للكروت والباقات
 - الأحمر #D94B4B وعائلته (#C83F3F , #8F3030 , #FFF0F0) مخصص حصريًا لسياق الطوارئ — أي حاجة حمرا في الواجهة معناها "خطر/تنبيه"، مفيش استخدام ثاني للأحمر خالص. ده قرار تصميم واعى: اللون بقى إشارة موثوقة (signal)، مش زخرفة.
 - الأصفر/الكهرماني (amber-50/600) لمستوى الخطورة "moderate" وبانر تذكير إكمال البروفايل — تحذير خفيف، مش إنذار.
- الملاحظة المهمة: الألوان دي مكتوبة كـ Tailwind arbitrary values مباشرة في ال JSX (زي bg-[#19A878]) في أغلب المكونات، مش كمتغيرات CSS من الأول. ده أثر على إزاي اتعمل الدارك مود — شوف الجزء الجاي.

■ 2.2 الدارك مود — إزاي متطبق تقنيًا بالظبط

المصدر: frontend/src/App.css + frontend/src/theme/ThemeProvider.jsx .

• آلية الحفظ والتفعيل (ThemeProvider.jsx)

1. القراءة الأولى: localStorage["najda-theme"] لو موجودة، وإلا: window.matchMedia("(prefers-color-scheme: dark)") (تفضيل نظام التشغيل).
2. useEffect بيحط النتيجة على عنصر <html> بطريقتين سوا:
 - document.documentElement.dataset.theme = theme → يعني <html data-theme="dark">
 - document.documentElement.style.colorScheme = theme → بيوري المتصفح نفسه (scrollbars, form controls) الافتراضية) إن الصفحة دارك.
3. فيه listener على matchMedia يغير الثيم تلقائي لو المستخدم غير تفضيل النظام وهو لسه ماخوش قرار يدوي (يعني localStorage فاضية) — أول ما يدوس التوجل، القرار اليدوي بيبقى له الأولوية للأبد.
4. toggleTheme() بيقلب بين light / dark ويخزن في localStorage فورًا.

• آلية الألوان (App.css) — CSS Variables (Design Tokens)

فيه `:root` بيعرّف مجموعة متغيرات (`--theme-page` , `--theme-surface` , `--theme-text` , `--theme-border` , `--` , `theme-accent` , `--theme-danger-soft` ... إلخ) بقيم فاتحة، و `:root[data-theme="dark"]` بيعيد تعريف نفس المتغيرات بقيم غامقة. المتغيرات دي مربوطة مباشرة بال `data-theme` attribute اللي `ThemeProvider` بيحطه.

المشكلة اللي اتحلت بطريقة غير تقليدية: أغلب المكونات مكتوبة بألوان hex صريحة (`bg-[#F8FAFB]` , `text-[#182B3A]` ...) مش بال `variables` دي مباشرة — يعني تغيير ال `data-theme` وحده ماكانش هيغيّر لون أي حاجة. الحل اللي اتعمل في `App.css` (سطر 217 لحد

1. هو طبقة جسر (bridge layer): `attribute selectors` بتمسك كل كلاس Tailwind بلون

معين وتستبدل قيمته تحت `:root[data-theme="dark"]` فقط، مثلاً:

CSS

```
:root[data-theme="dark"] :is(
  [class~="bg-[#F8FAFB]"],
  [class~="bg-[#F8FCFA]"],
  [class~="bg-[#FAFCFB]"]
) {
  background-color: var(--theme-page);
}
```

يعني في الدارك مود، أي عنصر عليه كلاس `bg-[#F8FAFB]` (أو أي كلاس تاني في القائمة) بيتحول لونه لقيمة `--theme-page` الغامقة، من غير ما نلمس أي سطر JSX. نفس المنطق مطبق على الخلفيات، النصوص، الحدود، ال `hover states`، والظلال (shadows) — القوائم دي مبنية يدويًا من كل قيم الهيكل المستخدمة فعليًا في المكونات وقت بناء الفيتشر.

ليه اتعمل كده مش **refactor** كامل لكل مكون؟ لأن المشروع كان شغال بالكامل بألوان مكتوبة مباشرة قبل ما الدارك مود يتضاف (Codex) بنى الدارك مود في `worktree` منفصل — `worktrees/dark-mode`. بالتوازي مع باقي الشغل، فالحل الأسرع والأمن كان جسر CSS يغطي كل قيمة موجودة فعليًا بدل ما تتلمس عشرات الملفات وتخاطر بكسر حاجة قبل الديدلاين. ده **trade-off** واعى: أسرع للتسليم، لكن أي لون هيكس جديد يتضاف بعدين في مكون لازم يتضاف يدويًا لقائمة الجسر في `App.css` وإلا هيفضل زي ما هو في الدارك مود (نسيان ده هو بالضبط سبب باج زرار جوجل — قصة حرب كاملة في قسم 5).

RTL والدارك مود مع بعض: فيه قواعد منفصلة تحت `:root[dir="rtl"]` (مش `data-theme`) بتقلب اتجاهات ال padding/border/position الفيزيائية (`.pl-10` , `.border-r` , `.rtl-flip` ...) لأن Tailwind classes دي `physical` مش `logical`. الاتنين (dark + rtl) مستقلين تمامًا عن بعض ويبشتغلوا مع بعض من غير تعارض لأنهم بيمسكوا خصائص مختلفة (ألوان ضد اتجاهات).

• 2.3 الثنائية اللغوية (عربي/إنجليزي)

المكتبة: `react-i18next` (v17) + `i18next` (v26). الملفات:

- `frontend/src/i18n/index.js` — الإعداد الرئيسى، بيسجل موردين (`en` , `ar`)
- `frontend/src/i18n/locales/ar.js` و `en.js` — 405 و 410 سطر على التوالي، namespaces زي `navigation` , `home` , `auth` , `signup` , `profile` , `chat` , `emergency` , `completeProfile` ...
- `frontend/src/i18n/language.js` — منطق كشف اللغة والاتجاه، فاضل تمامًا من أي تبعية على React (pure functions), عشان تتختبر بسهولة

كشف اللغة (detectLanguage): أولوية لـ `localStorage["najda-language"]` (اختيار المستخدم اليدوي)، وإلا `navigator.languages[0]` (لغة المتصفح)، وإلا `"en"` ك fallback. اللغة ستتطبع ك `languageOnly` (يعني `ar-EG` بترجم لـ `ar` مش عملة لغة منفصلة).

RTL/LTR: `getDirection(language)` بترجع `"rtl"` للعربي و `"ltr"` للإنجليزي، ومكون `LanguageSync.jsx` ييحط النتيجة على `document.documentElement.dir` و `.lang` — ويبعمل sync فوري عند التحميل (استدعاء مباشر مش جوه `useEffect` بس) وبعدين بيسمع لحدث `i18n.on("languageChanged", ...)` لأي تغيير لاحق.

تفاصيل حساسة اتعملت بعناية:

- كل مكون بيعرض بيانات نصية من المستخدم (اسم، رسالة شات) عليه `dir="auto"` مش `dir="ltr"` ثابت — عشان لو المستخدم كتب عربي جوه واجهة إنجليزي (أو العكس) النص يتحاذى صح.
- أرقام (عداد الطوارئ، الوقت، تواريخ) بتنسق بـ `Intl.DateTimeFormat / Intl.NumberFormat` مع `getFormattingLocale(language)` (`ar-EG` أو `en`) — يعني العداد بيظهر أرقام هندية/عربية فعلية لما اللغة عربي، مش أرقام إنجليزية بمجرد ترجمة الكلام حواليتها.
- أرقام التليفون والإيميلات دايماً `dir="ltr"` صريح حتى في الواجهة العربية (مش `auto`) لأنها مش نص طبيعي.
- `rtl-flip` كلاس CSS بيعكس أيقونات السهم (`transform: scaleX(-1)`) تحت `dir="rtl"` — عشان سهم "قدام" يفضل بصرياً صح في الاتجاهين.
- فيه اختبارات مخصصة (`frontend/tests/i18n/*.test.js`): `language.test.js`, `resources.test.js`, `startup.test.js`, `api-error.test.js` — بتأكد إن كل ال keys موجودة في اللغتين وإن مفيش مفتاح ناقص (parity check).
- ال backend نفسه بيعكس لغة المستخدم في التحية وال triage والمكالمة الحية (مذكور في `PROJECT-STATE.md` كومت `4795600`) — يعني لو المستخدم كتب بالإنجليزي، الرد بيرجع إنجليزي حتى لو الواجهة عربي.

3 مكونات ال UX الحرجة

3.1 بادج الخطورة (Risk Badge)

دالة `riskBadgeClass()` في `Chat.jsx` (سطر 55-65):

المستوى	اللون
<code>high / emergency</code>	أحمر (<code>bg-red-100 text-red-700</code>)
<code>moderate</code>	كهرماني (<code>bg-amber-100 text-amber-700</code>)
<code>low</code> (وأي حاجة تانية)	أخضر (<code>bg-emerald-100 text-emerald-700</code>)

نفس المنطق (بألوان مختلفة شوية) متكرر في `EmergencyHistory.jsx` (`RISK_META`) عشان يبقى متنسق عبر الشاشتين. البادج ده بيظهر في **مكانين مختلفين**: جوه ال `emergency strip` (شريط علوي أحمر ثابت وقت وضع الطوارئ)، وجوه ال header العادي للشات لو أي رسالة عادية طلعت خطورة عالية (زي ما شرحنا فوق — الطوارئ مش مقصورة على `/emergency` بس).

■ 3.2 بانر تذكير إكمال البروفايل (ProfileReminderBanner.jsx)

ده منفصل تمامًا عن بوابة `/complete-profile` — البوابة إجبارية لجهة اتصال الطوارئ بس، والبانر ده تذكير غير مانع (dismissible) لو باقي حقول (تاريخ ميلاد/جنس/فصيلة دم) لسه فاضية. بيظهر أعلى `/profile` وأعلى `/chat` (فوق منطقة الرسائل).

- الحالة `dismissed` بتتخزن في `sessionStorage` (مش `localStorage`) — يعني لو قفلته، بيختفي للجلسة دي بس وبيرجع يظهر تاني لما تفتح تاب/سيشن جديدة. قرار مقصود: تذكير لطيف متكرر، مش إخفاء دائم.
- بيقدّر ياخد `profile` جاهزة ك prop (زي ما بيحصل في `Profile.jsx` عشان مايكررش ال `fetch` أو يجيبها بنفسه (GET `/profile/me`) لو محدش داله حاجة.
- فيه تعليق صريح في الكود (سطر 86-89) بيشرح ليه استخدم `<div>` مش `<p dir="auto">`: اختبار Playwright بيحدد فقاعات الشات بـ `main p[dir="auto"]`، ولو البانر استخدم نفس التاج كان هيتحسب غلط كرسالة شات.

■ 3.3 كارت العداد التنازلي + السرينة + الاهتزاز

ده أهم قطعة UX في المشروع، وسبب وجودها بسيط: حالة طوارئ طبية لازم تكون "مستحيل تتفوت". لو المستخدم مغمي عليه جزئيًا أو مش قادر يمسك التليفون، مجرد نص على الشاشة مش كفاية.

التسلسل (Chat.jsx سطر 242-400):

1. لما `emergency_event.escalation_status` تبقى `"alert_pending"`، بيتفعل تلقائيًا:
 - عداد 60 ثانية بيتحسب كل ثانية (`useEffect` مع `setTimeout` متسلسل، سطر 365-381)
 - `startSiren()` (سطر 259-300): صوت Web Audio API يبني بيه صفارة إسعاف حقيقية — oscillator رئيسي `(sine, 0.9Hz) + (880Hz, triangle)` بيأرجح التردد $280\text{Hz} \pm$ ، يعني صوت "يعلى وينزل" زي صفارة الإسعاف الحقيقية مش نغمة تنبيه عادية ثابتة.
 - اهتزاز (`navigator.vibrate`) بنمط `[400, 180, 400, 180, 500]` بيتكرر كل 2 ثانية — مهم للموبايل لو الصوت مقفول أو السماعه مش قريبة.
2. لو دُست "أنا بخير" (`handleRespond`) قبل ما الوقت يخلص → `POST /emergency/events/{id}/respond` → الحالة بتبقى `resolved`، والسرينة والاهتزاز بيوقفوا فورًا (`stopSiren()` في ال `cleanup` بتاع ال `useEffect` ، سطر 321-330).
3. لو الوقت خلص من غير رد → `handleAutoEscalate()` → `POST /emergency/events/{id}/escalate` → السيرفر بيرجّع بيانات جهة الاتصال، والواجهة بتعرض "بيتم إخطار [الاسم] — [الرقم]".
4. مقاومة الأخطاء: لو ال `escalate call` فشل (شبكة مثلاً) والسبب مش `"already"` (يعني الحالة مش خلصت أصلًا في مكان تاني) — بيعيد المحاولة تلقائيًا كل 5 ثواني بدل ما يجمد العداد. تعليق صريح في الكود (سطر 354-359) بيقول إن ده `"safety-critical"` فمينفّش يفشل بصمت.
5. تبديل الجلسة (فتح شات تاني من السايديبار) بيغفل حالة الطوارئ المخفية (`setEmergencyEvent(null)` في `loadSession`) عشان عداد قديم ماي فضلش شغال في الخلفية ويصعد حدث انتهى.

■ 3.4 ال quick-prompt في مسار الطوارئ — ليه مفيش فورم يحبس حد

في `/emergency` (`EmergencyMode.jsx`)، لو المستخدم داخل بدون جهة اتصال طوارئ محفوظة، بيظهر كارت صغير ("Quick emergency contact") فيه حقلين بس (اسم اختياري + رقم إجباري) وزرار حفظ. لكن جواره مباشرة زرار "تخطي" (`bypassLink`)

بيوديك للشات على طول من غير ما تحفظ حاجة خالص.

الفلسفة هنا معاكسة تمامًا لـ `/complete-profile`: هناك جهة الاتصال إجبارية (`canSubmit`) مقفول لحد ما تتملى) لأن المستخدم مش في أزمة وقت التسجيل العادي — وقت مناسب تاخذ منه دقيقة. لكن هنا، في مسار الطوارئ الفعلي، أي عائق إضافي بين المستخدم والمساعدة ممكن يكلف وقت حرج. لو حبسناه في فورم إجباري وقت أزمة حقيقية، النظام نفسه بيبقى الخطر. فالتصميم قرر: اعرض الخيار، خليه سريع لو عايز يملاه، لكن سيبه باب مفتوح للخروج فورًا لأي حاجة.

4 قاعدة البيانات

المحرك: PostgreSQL سحابي على Neon (serverless Postgres). الكود بينزل تلقائيًا لـ SQLite ملف (`sqlite:///./cacheai.db`) — نفس الـ ORM models شغالة على الاثنين بفضل `PortableJSON` (شوف تحت). الاتصال والتحويل في `backend/database.py`.

الـ ORM: SQLAlchemy 2.0 style (`Mapped[...]`, `mapped_column`) — الموديلز كلها في `backend/models.py` (337 سطر).

4.1 الجداول

• users

- `id` (PK), `name`, `email` (unique, indexed), `password_hash` (nullable — فاضي لحسابات Google), `auth_provider` ("local"/"google"), `provider_id` (unique, nullable — Google II sub), `created_at`, `updated_at`
- علاقات: `patient_profile` (1:1), `emergency_contact` (1:1), `chat_sessions` (N:1), `emergency_events` (1:N) — كل العلاقات دي `cascade="all, delete-orphan"`, يعني مسح مستخدم ييمسح كل بياناته المرتبطة تلقائيًا (مفيش يتامى في الداتايز).

• patient_profiles

- `id` (PK), `user_id` (FK → users, unique), `patient_id` (nullable, unique), `date_of_birth`, `gender`, `blood_type`, `chronic_conditions` (array نصوص — `PortableJSON` timestamps).
- **ليه `patient_id` nullable و unique مع بعض؟** عشان مستخدم من غير رقم ملف طبي (أغلب الناس اللي بتسجل من البيت) يقدر يسيبه فاضي بدون ما يتصادم مع مستخدم ثاني فاضي برضه — في SQL، قيم `NULL` متعددة في عمود `UNIQUE` مسموحة ومتتصادمش مع بعض (بعكس `""` فاضية اللي بتتصادم). ده بالضبط سبب باج حقيقي حصل — قصة كاملة في قسم 5.

• emergency_contacts

- `id` (PK), `user_id` (FK → users, unique = 1:1), `name`, `phone`, `email` (الثلاثة nullable), timestamps.
- جدول منفصل عن `patient_profiles` عمدًا (مش عمود جوه users) — عشان يتقدر يتعمله upsert مستقل (PUT `/profile/emergency-contact`) بدون ما يلمس باقي البروفايل، ولأن منطق "موجود ولا لا" (البوابة) بيحتاج يتفحص بمعزل عن باقي الحقول.

• chat_sessions

- `chat_type` · (nullable) — يتحدد تلقائي من أول رسالة) `title` · (FK → users, nullable) `user_id` · (PK) `id` · timestamps · ("normal"/"emergency", default "normal")
- علاقات: `messages` (N, cascade delete:1), `emergency_events` (1:N, cascade delete)

• messages

- `sources` · (Text) `content` · ("user"/"assistant") `sender` · (FK) `chat_session_id` · (PK) `id` · (nullable,) `risk_level` · ({title, org, url} من nullable — array, PortableJSON) · `created_at` · ("low"/"moderate"/"high"/"emergency"
- `sources` و `risk_level` انضافوا بعد الجداول الأساسية (شوف الهجرات تحت) — كل رد من المساعد يسجل مصادره ومستوى خطورته مع الرسالة نفسها، يعني السجل التاريخي كامل حتى لو ال RAG index اتغير بعدين.

• emergency_events

- `id` · (PK) `user_id` · (FK) `chat_session_id` · (FK) `condition` · (nullable —) `started_at` · ("default "low) `risk_level` · ("stroke"/"chest_heart"/"breathing"/"unknown" — الداتا بيز نفسها بتحط الوقت، مش الكود) `timer_seconds` · (default 60) `escalation_status` · (nullable) `resolved_at` · ("default "monitoring) `responded_at`
- **State machine** صريحة على `escalation_status` : `monitoring` → `alert_pending` → `resolved` (أو `escalated`). القفل مطبق في `backend/routers/emergency.py`
- `/respond` مسموح بس من `monitoring` أو `alert_pending` (409 لو الحالة خلصت خلاص)
- `/escalate` مسموح بس من `alert_pending` (409 لو حاول يصعد حالة already resolved)
- الحالتين النهائييتين (`escalated`, `resolved`) **نهائيتين فعلاً** — مفيش مسار بيرجعهم.
- `chat.py` سطر 259-263 كمان بيتأكد إن رد عالي الخطورة **معا ديش يفتح** حدث خلص خلاص `(event.escalation_status in ("monitoring", "alert_pending"))` قبل أي تحديث).
- ده كان أحد إصلاحات مراجعة Codex (مذكور في `PROJECT-STATE.md`) — قبل كده كان ممكن event منتهي "يرجع يفتح" بغلط.

■ 4.2 PortableJSON — له فيه type مخصص

PYTHON

```
class PortableJSON(TypeDecorator):
    impl = JSON
    def load_dialect_impl(self, dialect):
        if dialect.name == "postgresql":
            return dialect.type_descriptor(JSONB())
        return dialect.type_descriptor(JSON())
```

على PostgreSQL (Neon) يستخدم `JSONB` الفعلي (indexed, أسرع وأقوى للاستعلامات)، وعلى SQLite (التطوير المحلي / fallback بدون داتا بيز) بيرجع لـ `JSON` عادي لأن SQLite أصلاً معندوش `JSONB`. الفائدة: نفس كود ال ORM بيشتغل identical على البيئتين من غير `if` منتشر في كل مكان بيستخدم `JSON`.

■ 4.3 Neon (Postgres) + Qdrant vectors — ليه منفصلين

Neon مسؤول عن **البيانات العلائقية المنظمة**: مستخدمين، بروفايلات، محادثات، أحداث طوارئ — بيانات لها schema ثابت وعلاقات (foreign keys) و logic ترانزاشن (commit/rollback) واضح.

Qdrant مسؤول عن **البحث الدلالي (semantic search) في المحتوى الطبي** — المصادر الطبية (WHO/AHA-ASA/NHS/CDC) — متحولة ل embeddings (متجهات أرقام 768-بعد)، والبحث فيها يتم بمقارنة تشابه متجهات (cosine similarity) مش بمطابقة نص. الاتنين نوع بيانات ومنطق استعلام مختلف جوهريًا — تخزين متجهات جوه أعمدة Postgres عادي ممكن (فيه pgvector extension) لكن مش الخيار اللي اتأخذ هنا، والسبب العملي: Qdrant محرك متخصص لـ vector search بأداء أعلى وميزات جاهزة (hybrid search, reranking) مش متاحة بنفس السهولة لو بنيناها فوق Postgres.

تفصيلة مهمة تتقال للجنة لو سألوا: المشروع فيه نظامين RAG منفصلين فعليًا:

1. `backend/ai/rag.py` — Qdrant محلي (embedded, on-disk) جوه `backend/qdrant_data/`، بيستخدم `gemini-embedding-001` (dim-768)، وده الاحتياطي اللي شغال جوه مسار الـ "triage" الافتراضي.

2. محرك NAJDA (`app/retrieval.py`)، بورت 8001 منفصل) — Qdrant Cloud حقيقي (`url` + `api_key`)، بـ hybrid search + reranker (`CrossEncoder`) فوق 574 نقطة من 9 مصادر guidelines منضفة — ده بيتنادى بس للأسئلة "الإكلينيكية" الصريحة (كلمات مفتاحية زي "علاج"، "جرعة"، "بروتوكول" — `_CLINICAL_KEYWORDS` في `backend/ai/agent.py`). `backend/ai/agent.py` هو الراوتر اللي بيقرر مين يرد (`smalltalk/clinical/triage`) وبيرجع fallback نصي آمن لو الاتنين فشلوا — الشات عمره ما يرجع 500 بسبب طبقة الـ AI.

■ 4.4 الهجرات — `migrate_schema.py` بدون Alembic

المشروع مبيستخدمش Alembic (أداة الهجرات القياسية لـ SQLAlchemy). بدالها فيه `backend/migrate_schema.py` — سكريبت يدوي، بيضيف فقط ولا يسمح أبدًا:

- `COLUMN_DEFINITIONS`: dict بيوصف كل عمود جديد المفروض يتضاف لكل جدول، مع نوعه.
- ييفحص الأعمدة الموجودة فعليًا (`inspector.get_columns`) وبيضيف بس اللي ناقص (`ALTER TABLE ... ADD COLUMN`).
- **تفصيلة تقنية مهمة:** SQLite بيرفض `ALTER TABLE ADD COLUMN` بقيمة `DEFAULT` غير ثابتة (زي `DEFAULT CURRENT_TIMESTAMP`) لو الجدول فيه صفوف خلاص. الحل: السكريبت بيضيف جزء `DEFAULT` من تعريف العمود وقت الـ `ALTER` (`_DEFAULT_CLAUSE_RE`)، وبعدين بيعمل `UPDATE ... WHERE col IS NULL` منفصل (`BACKFILL_STATEMENTS`) يملأ القيم الافتراضية للصفوف القديمة يدويًا — كده الأمر شغال على SQLite و PostgreSQL بنفس الكود.
- بيتشغل مرة واحدة يدويًا (`python migrate_schema.py`) قبل ما تشغل السيرفر، مش تلقائي عند الإقلاع.

ليه بدون Alembic؟ قرار عملي مش مبدئي — تحت ضغط وقت المسابقة، وباعتبار الداتايز فيها بيانات production حقيقية على Neon (مش بيئة تجريبية تتمسح وتتبني من الصفر)، سكريبت مباشر "idempotent + additive-only" (يشغل أي عدد مرات وميغيرش حاجة موجودة) كان أسرع وأمن من تأسيس Alembic كامل (`migrations folder, versioning, autogenerate`) لمشروع بمرحلة التطور دي. الـ trade-off الواضح: مفيش (`rollback/downgrade()` رسمي، ومفيش تاريخ migrations مرقم — لو المشروع كبر، Alembic هيبقى الخطوة المنطقية الجاية.

`init_db.py` (5 أسطر) هو نظير أبسط: `Base.metadata.create_all(bind=engine)` بيبنى كل الجداول من الصفر لو مش موجودة (بيئة جديدة تمامًا). تفصيلة تستاهل الانتباه: الملف بيستورد بالاسم أربع كلاسات بس (`EmergencyContact, Message`).

from EmergencyEvent — (PatientProfile, ChatSession, User) مش مذكورة صراحة في سطر الاستيراد، لكن السطر ... models import بينقذ ملف models.py كامل، فكل الكلاسات المعرّفة فيه (بما فيها EmergencyEvent) بتتسجل تلقائيًا في Base.metadata بمجرد استيراد أي حاجة من الملف — فالجدول بيتعمل صح برضه.

5 قصص حرب حقيقية (تديك مصداقية قدام اللجنة)

5.1 باج patient_id الفاضي — 409 كانت بتقفل التسجيل فعليًا

اكتشفته (commit 3ef9ada) Playwright E2E suite. الفورم (SignupFormFields.jsx) كان بيعت patient_id: formData.patientId من غير أي تحقق — يعني لو المستخدم سايب الحقل فاضي، كانت بتتبعث "" (نص فاضي) للباك إند، مش null.

المشكلة: عمود patient_id في patient_profiles معرّف unique=True. في SQL، قيمة NULL مش بتتصادم أبدًا مع NULL ثانية تحت UNIQUE constraint (المعيار بيعتبر كل NULL "غير معروفة" ومش متساوية حتى مع نفسها) — لكن "" قيمة فعلية حقيقية، وبالتالي أول مستخدم يسجل من غير patient ID ياخد "", وأي مستخدم ثاني بعده يسجل من غير patient ID كمان يصطدم ب IntegrityError (unique violation) → السيرفر بيرجع 409. يعني التسجيل كان مكسور فعليًا لكل مستخدم ثاني وما بعده ما لم يكتب patient ID مميز يدويًا — وده أغلب الناس مابتعملوش وقت التسجيل الأول.

الإصلاح سطر واحد: patient_id: formData.patientId || null بدل الإرسال المباشر. الدرس الأهم هنا للجنة: الباج ده مكانش هيظهر بأي اختبار يدوي عادي (أول تسجيل بيعدي عادي تمامًا)، وده بالظبط سبب قيمة الـ E2E suite (38/38) اختبار — auth/profile/chat/emergency state machine (الي بتحاكي مستخدمين متعددين حقيقيين).

5.2 الصندوق الأبيض حوالين زرار جوجل في الدارك مود

باج بصري ظهر بعد إضافة الدارك مود: زرار "Continue with Google" (الي بيرسمه Google Identity Services نفسه جوه <iframe> منفصل، مش عنصر HTML بتاعنا) كان بيظهر جوه مستطيل أبيض وسط كارت غامق — بيبين واضح وبايظ بصريًا.

السبب الجذري (تعليق commit e4244d7) بيشرحه بدقة: ThemeProvider بيحط document.documentElement.style.colorScheme = "dark" على مستوى الصفحة كلها. لكن الزرار جوه <iframe> من origin مختلف تمامًا (accounts.google.com) وبيعرّف color-scheme بتاعه هو لوحده. Chromium عنده سلوك محدد: لو الـ color-scheme بتاع الصفحة الحاضنة (embedder) مايطابقش الـ color-scheme بتاع الـ iframe جواها، المتصفح بيرسم خلفية بيضاء صريحة خلف الـ iframe (كإجراء أمان بصري، مش باج في Google نفسها). يعني المشكلة مكانتش في تصميمنا ولا في GSI — كانت في تفاعل ضمني بين تفضيلين مختلفين للـ color-scheme عبر حدود origin.

الإصلاح: style={{ colorScheme: "light" }} على الـ <div> الحاوي مباشرة (مش على <html>) في Login.jsx و EmergencyAuth.jsx. كده الـ wrapper يقول للمتصفح "أنا فاتح هنا تحديدًا" فيتطابق مع الـ iframe جواه (Google) بترسم زرارها بـ filled_black theme وقت الدارك مود أصلًا، فمش محتاجة الخلفية البيضاء). اتحقق فعليًا بفحص الـ computed styles (مش بس "شكله بقى تمام") — iframe color-scheme: light تحت data-theme=dark بعد الإصلاح.

الدرس: باجات المتصفح عبر origin boundaries (زي CSS color-scheme مع iframes) نادرًا ما توضح نفسها من أول نظرة — محتاجة فحص computed styles فعلي، مش تخمين بصري.

6 أسئلة متوقعة من الحكام + إجابات جاهزة

1. ليه مفيش تطبيق موبايل (native app)؟

المشروع (React SPA) Progressive-friendly web app بيشتغل من أي متصفح بدون تنزيل — أهم حاجة وقت طوارئ إن الوصول يكون فوري، مش مربوط بمتجر تطبيقات أو تحديث. بناء (iOS + Android) native app كان هيضاعف وقت التطوير في نافذة مسابقة قصيرة، من غير قيمة إضافية حقيقية للمستخدم مقارنة بويب أب سريع. (ملاحظة صراحة: ال responsiveness على الموبايل مقيس ومكسور جزئيًا حاليًا — قرار مؤجل بوعي بسبب ضغط الوقت، مش حاجة اتنسيتها).

2. إزاي بتحموا البيانات الطبية الحساسة؟

- المصادقة عبر (HS256) JWT بعمر صلاحية أسبوع (`ACCESS_TOKEN_EXPIRE_MINUTES=10080`)، والباسورد متخزن مهشّر بـ `bcrypt` (`passlib`) — مفيش باسورد نص صريح في الداتا بيز أبدأ.
- كل endpoint بيانات شخصية محمي بـ `get_current_user` (`Depends`) بيستخرج المستخدم من التوكن، وكل استدعاء بيانات مفلتر بـ `user_id == current_user.id` — مستخدم مايقدرش يشوف بيانات مستخدم ثاني حتى لو عرف ال ID بتاعه (IDOR protection على مستوى كل query).
- Google Sign-In: التوكن بيتفحص فعليًا ضد Google (`verify_oauth2_token`) بالـ `audience` المضبوط، وبيتأكد من `iss` و `email_verified` قبل ما يوثق بيه — مش مجرد "استقبال وتصديق".
- `.env` (فيها الأسرار: `DATABASE_URL` , `SECRET_KEY` , `GEMINI_API_KEY` ...) متشالة من git tracking (`.gitignore`)، ولو سألوا بصراحة: فيه باسورد Neon قديم اتسرب في تاريخ git عام قبل كده ولسه معلق تدويره — إجابة صادقة أحسن من إخفاء.

3. ليه Postgres مش MongoDB (أو NoSQL عموماً)؟

بياناتنا **علائقية بطبيعتها**: مستخدم له بروفایل واحد، جهة اتصال واحدة، محادثات متعددة، كل محادثة رسائل متعددة، كل حدث طوارئ مربوط بمستخدم ومحادثة معينة — foreign keys وقيود `NOT NULL / UNIQUE` بتضمن سلامة البيانات دي على مستوى الداتا بيز نفسها (مش بس على مستوى كود التطبيق). حاجة زي منع تكرار `patient_id` أو ضمان إن جهة اتصال الطوارئ ليها مستخدم واحد بالظبط — ده بالظبط اللي ال relational constraints معموله عشانه. بيانات شبه-منظمة زي `chronic_conditions / sources` اتحلت بعمود `JSONB` جوه Postgres نفسه (مش محتاجين داتا بيز ثانية كاملة عشانها).

4. إزاي الدارك مود شغال بالظبط؟

(جاوب بالتفصيل اللي في قسم 2.2 فوق — النقطة الأهم اللي تتقال: مش مجرد `prefers-color-scheme`، فيه تخزين تفضيل يدوي في `localStorage` بألوية أعلى من تفضيل النظام، و `data-theme` + CSS variables + attribute + طبقة جسر بتربط قيم Tailwind الصريحة بالمتغيرات).

5. الثنائية اللغوية — إزاي بتضمنوا مفيش نص إنجليزي ناسي وسط شاشة عربي؟

فيه اختبارات آلية (`frontend/tests/i18n/resources.test.js`) بتأكد إن كل مفتاح ترجمة موجود في الملفين (`ar.js` و `en.js`) بنفس البنية — لو حد ضاف مفتاح في واحد ونسي الثاني، الاختبار بيفشل. وده منفصل عن اختبار الـ E2E اللي بيتأكد إن الواجهة شغالة فعليًا باللغتين.

6. إيه اللي بيحصل لو النظام مايعرفش يجاوب سؤال طبي (الـ AI فشل)؟

مصمم إنه عمره ما يرمي **exception** توصل للمستخدم. فيه سلسلة fallback كاملة: Groq (النموذج الأساسي) → Groq بنموذج ثاني (لو الكوطة اليومية خلصت — الكوطات per-model مش عامة) → Gemini (احتياطي) → نص عربي آمن ثابت (`prompts.FALLBACK_ANSWER`) لو الكل فشل. النتيجة: `POST /chat/sessions/{id}/messages` عمرها ما ترجع 500 بسبب طبقة الذكاء الاصطناعي.

7. إزاي بتضمنوا إن حالة طوارئ فعلية متفتوش لو المستخدم ماردش؟

عداد 60 ثانية + صفارة صوت (Web Audio API، نمط صعود/هبوط زي سيارة إسعاف حقيقية) + اهتزاز متكرر كل ثانيتين — الثلاثة سوا، مش بس نص على الشاشة. لو الوقت خلص من غير رد، تصعيد تلقائي لجهة الاتصال (`escalate` endpoint) بمحاولات retry تلقائية كل 5 ثواني لو فشل الطلب الأول — مفيش سيناريو "العداد وصل صفر ومفيش حصل حاجة".

8. ليه المستخدم مش مجبر يملئ بياناته الطبية كاملة عند التسجيل؟

لأن إجبار مستخدم في لحظة تسجيل عادية على فورم طويل بيقلل معدل الإتمام (drop-off)، وأهم حاجة فعلاً وقت أزمة هي **جهة اتصال طوارئ صالحة** — دي بس اللي إجبارية (`/complete-profile` / قسم 1). باقي البيانات (فصيلة دم، أمراض مزمنة...) قيمتها إضافية مش حرجة لتشغيل النظام، فهي اختيارية بالكامل ويمكن تتضاف أي وقت من (`/edit-profile`)، مع بانر تذكير لطيف غير مانع.

9. إيه الفرق بين البوابة الإجبارية والفورم السريع وقت الطوارئ؟ مش تناقض؟

لا — فلسفتين مختلفتين لسياقين مختلفين تمامًا (تفصيل كامل في قسم 3.4). وقت التسجيل العادي الوقت متاح، فبنجبر بيانات أمان أساسية. وقت أزمة حقيقية (`/emergency`)، أي عائق إضافي خطر — فالفورم موجود كخيار سريع بس مع زرار "تخطي" واضح دايماً. القرار اتأخذ بوعي إن safety-critical UX لازم يفضل السرعة على الاكتمال وقت الأزمة الفعلية.

10. ليه فيه نظامين RAG منفصلين (Qdrant محلي و Qdrant Cloud)؟

مش تصميم من الأول — نتيجة دمج شغلين متوازيين (فريق ثاني بنى محرك NAJDA بـ hybrid retrieval + reranker على Qdrant Cloud بمصادر منضفة، بينما `backend/ai/rag.py` كان already موجود كطبقة احتياطية محلية). الراوتر في `agent.py` بيوّج الأسئلة الإكلينيكية الصريحة (كلمات زي "علاج"/"جرعة"/"بروتوكول") لمحرك NAJDA الأدق، والباقي (تحية، أسئلة عامة، triage) بيعدي على المسار الافتراضي. لو NAJDA وقع أو رجع ungrounded، بيرجع تلقائي (fallback) للمسار الافتراضي — مفيش نقطة فشل واحدة.

11. إزاي بتضمنوا إن مستخدم مايشوفش بيانات مستخدم تاني؟

كل استعلام في كل router (`profile.py` , `emergency.py` , `chat.py`) بيفلتر صراحة بـ `user_id ==` `current_user.id` (أو عبر علاقة `owned session/event`) — مفيش endpoint بياخد `user_id` ك parameter من العميل ويثق فيه؛ الهوية الوحيدة المعتمدة جايه من التوكن نفسه (`get_current_user`). حتى الوصول لمحادثة معينة بـ ID بيتأكد الأول إنها مملوكة للمستخدم الحالي قبل ما يرجعها (`_get_owned_session`), وإلا 404.

12. ليه مفيش Alembic للهجرات؟

قرار عملي مش مبدئي — الوقت المتاح والبيانات الحقيقية على Neon (مش بيئة تجريبية) خلّوا سكريبت "additive-only" بسيط (`migrate_schema.py`) الخيار الأسرع والأمن آنياً. Trade-off واضح ومعروف: مفيش downgrade رسمي ولا ترقيم إصدارات — لو المشروع كبر بعد المسابقة، Alembic هو الخطوة المنطقية الجاية. (تفصيل كامل في قسم 4.4).

هذا المستند اتبنى بقراءة الكود الفعلي فقط (مفيش افتراضات) بتاريخ 20-08-2026، من: `frontend/src/pages/*` , `frontend/src/App.jsx` , `frontend/src/App.css` , `frontend/src/i18n/*` , `frontend/src/theme/*` , `frontend/src/components/*` , `backend/init_db.py` , `backend/migrate_schema.py` , `backend/database.py` , `backend/models.py` , `docs/PROJECT-` , `app/retrieval.py` , `backend/ai/{agent,rag}.py` , `backend/routers/{auth,chat,emergency,profile}.py` , `STATE.md`